

CrewAI

실무 가이드 · 지식 점검

역할 기반 멀티 에이전트 프레임워크 — 핵심 개념 · 패턴 · 코드 · 함정

LangGraph 경쟁 솔루션 · Crews + Flows 아키텍처

CrewAI 기준 (2026년 6월) · 대상: 풀스택 / MLOps / DevOps 엔지니어

난이도: 실무 핵심 · 본문 한국어 + 코드/용어 영문

1. CrewAI란 무엇이고 언제 쓰는가
2. 핵심 멘탈 모델: Crews vs Flows
3. Agent — 역할 기반 작업자
4. Task — 작업 정의와 구조화 출력
5. Crew — 조립과 실행
6. Process: Sequential vs Hierarchical
7. Flows — 이벤트 기반 오케스트레이션
8. Tools — 내장·커스텀 도구
9. Memory와 Knowledge
10. 실행: kickoff·입력 보간·반복·비동기
11. 설정: YAML + CrewBase vs 순수 파이썬
12. 학습·테스트·관측·배포
13. CrewAI vs LangGraph 비교
14. 자주 하는 실수 모음
15. API 빠른 레퍼런스

지식 점검 퀴즈 (객관식 20문항)

정답 및 해설

1 CrewAI란 무엇이고 언제 쓰는가

CrewAI는 **역할(role) 기반 멀티 에이전트**를 빠르게 조립하는 파이썬 프레임워크다. "리서처·작가·검수자" 같은 역할을 가진 에이전트들을 하나의 **크루(Crew)**로 묶어 협업시키는 직관적 추상화가 핵심이다. LangChain에 의존하지 않는 **독립적·경량 프레임워크**로 처음부터 다시 작성됐고, 적은 코드로 멀티 에이전트를 띄울 수 있어 학습 곡선이 완만하다.

두 개의 빌딩블록 — Crews와 Flows

CrewAI는 두 축으로 구성된다. **Crews**는 역할 기반 에이전트들이 **자율적으로 협업**하는 단위(창발적, 적은 제어). **Flows**는 그 위에서 **이벤트 기반으로 실행 순서·분기·상태·영속성을 정밀 제어**하는 오케스트레이션 계층이다. "Crew = 일하는 작업자, Flow = 순서와 실패 처리를 아는 관리자"로 이해하면 된다. 둘은 함께 조합된다.

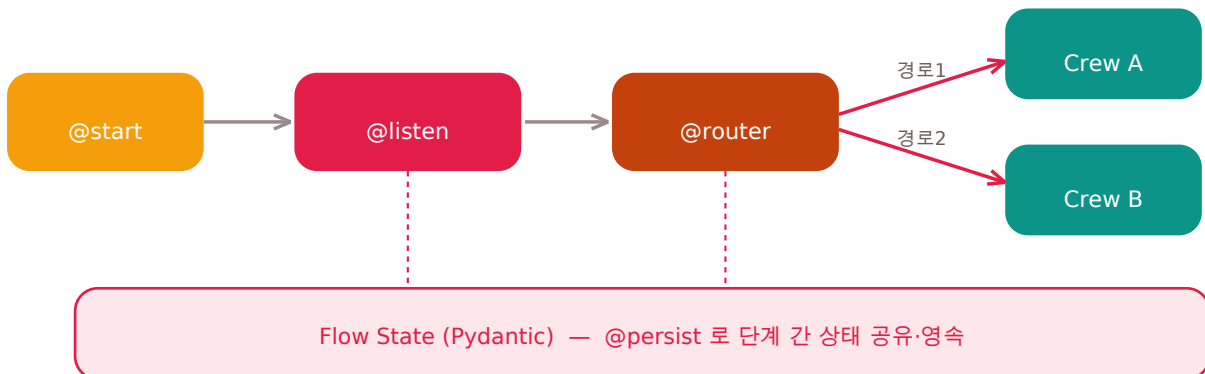
상황	권장
역할 분담이 명확한 협업(리서치→작성→검수)	Crew (sequential)
빠른 프로토타이핑·적은 코드	CrewAI Crew
단계별 정밀 제어·분기·실패 복구·상태 영속	CrewAI Flows
그래프 수준의 복잡한 순환/조건 제어가 필요	LangGraph 고려
크루(자율)와 플로우(제어)를 함께	Flow가 여러 Crew를 호출

한 줄 정의

CrewAI = "역할 기반 에이전트들의 협업(Crews)을, 이벤트 기반 오케스트레이션(Flows)으로 감싸 운영하는 프레임워크".

2 핵심 멘탈 모델: Crews vs Flows

- **Agent** — 역할·목표·배경(backstory)·도구를 가진 자율 작업자(LLM 구동).
- **Task** — 에이전트에게 맡기는 구체적 작업. 설명과 기대 산출물을 정의.
- **Crew** — 에이전트+태스크를 묶어 Process로 실행 순서를 결정하는 컨테이너.
- **Flow** — @start/@listen/@router로 단계를 잇고 State를 공유하는 이벤트 기반 흐름.



▲ Flow 오케스트레이션: @start → @listen → @router로 분기하여 여러 Crew를 호출, 공유 State(Pydantic)는 @persist로 영속.

Crews와 Flows의 차이

Crew는 태스크를 순서대로(또는 매니저가 위임) 실행하지만 "A 실행 → 결과 확인 → B나 C 중 선택" 같은 조건 분기를 직접 제어하기 어렵다. Flow는 바로 그 정밀 제어와 상태 관리를 제공한다. 프로토타입은 Crew로 시작하고, 운영에서 제어가 필요해지면 Flow로 감싸는 패턴이 일반적이다.

3 Agent — 역할 기반 작업자

Agent는 **role(역할)**·**goal(목표)**·**backstory(배경)** 세 요소로 페르소나를 정의한다. 이 서술이 프롬프트로 들어가 행동을 좌우하므로 구체적으로 쓸수록 좋다. 도구·LLM·위임 허용 여부 등을 함께 지정한다.

```
from crewai import Agent

researcher = Agent(
    role="Senior Researcher",
    goal="Collect accurate, up-to-date facts about {topic}",
    backstory="A rigorous researcher who verifies every source.",
    tools=[search_tool],
    llm="gpt-4.1",
    allow_delegation=False, # may delegate subtasks to other agents
    max_iter=15, verbose=True,
)
```

- **allow_delegation** — True면 다른 에이전트에게 하위 작업을 위임/질문 가능.
- **tools** — 에이전트가 쓸 도구 목록. 태스크 단위로도 지정 가능.
- **reasoning.max_iter.max_rpm** — 추론 단계, 반복 한도, 분당 호출 제한.

4 Task — 작업 정의와 구조화 출력

Task는 **description(무엇을)**과 **expected_output(어떤 형태로)**를 명시하고 담당 agent를 지정한다. context로 이전 태스크들의 결과를 입력으로 연결하고, Pydantic으로 구조화 출력을 강제할 수 있다.

```
from crewai import Task
from pydantic import BaseModel

class Report(BaseModel):
    title: str
    bullets: list[str]

research = Task(description="Research {topic}", expected_output="Top 5 facts",
                 agent=researcher)

write = Task(
    description="Write a report from the research",
    expected_output="Title + 5 bullets",
    agent=writer,
    context=[research], # feed prior task output as input
    output_pydantic=Report, # enforce structured output
    # human_input=True, # insert a human review step
)
```

- **context** — 선행 태스크 결과를 명시적으로 주입(의존 관계).
- **output_pydantic/output_json/output_file** — 구조화·파일 출력.

- guardrail — 산출물 검증/재시도 규칙. human_input=True — 사람 검토 삽입.
- async_execution=True — 해당 태스크를 비동기로 실행.

5 Crew — 조립과 실행

Crew는 에이전트와 태스크를 모아 process로 실행 방식을 정한다. memory=True로 메모리를, kickoff(inputs=...)로 실행을 시작한다. description의 {placeholder}는 inputs 값으로 보간된다.

```
from crewai import Crew, Process

crew = Crew(
    agents=[researcher, writer],
    tasks=[research, write],
    process=Process.sequential, # or Process.hierarchical
    memory=True,
    verbose=True,
)
result = crew.kickoff(inputs={"topic": "LangGraph"}) # fills {topic}
print(result.raw) # final text
print(result.pydantic) # structured output (if set)
```

6 Process: Sequential vs Hierarchical

Process	동작	제어자	적합
Sequential	태스크를 정의 순서대로 차례로 실행	정해진 순서	선형 파이프라인
Hierarchical	매니저가 역할 기반으로 동적 위임·조율	매니저 에이전트	순서 미정·복잡 분담

Hierarchical은 CrewAI가 자동 생성하거나 지정한 **매니저 에이전트**가 다른 에이전트에게 작업을 위임한다. 이때 manager_llm 또는 manager_agent를 반드시 지정해야 한다.

```
crew = Crew(agents=[...], tasks=[...],
            process=Process.hierarchical,
            manager_llm="gpt-4.1") # or manager_agent=custom_manager
```

7 Flows — 이벤트 기반 오케스트레이션

Flow는 @start()(진입점), @listen(이전단계)(선행 완료 시 실행), @router()(반환 문자열로 분기) 세 데코레이터가 핵심이다. 모든 단계가 공유하는 **State**는 Pydantic 모델이며 전역/인자 전달 없이 읽고 쓴다.

```
from crewai.flow.flow import Flow, start, listen, router
from pydantic import BaseModel
```

```
class State(BaseModel):
    topic: str = ""
    score: int = 0

class ContentFlow(Flow[State]):
    @start()
    def fetch(self):
        self.state.topic = "CrewAI"

    @listen(fetch)
    def evaluate(self):
```

```

self.state.score = 7          # write to shared state

@router(evaluate)
def route(self):
    return "deep" if self.state.score >= 7 else "shallow"

@listen("deep")
def deep_dive(self):
    return ResearchCrew().crew().kickoff(inputs={"topic": self.state.topic})

ContentFlow().kickoff()

```

- `and_(a, b)/or_(a, b)` — 여러 선행 단계의 결합 조건으로 `@listen` 트리거.
- `@persist` — 클래스/메서드 레벨에 붙여 State를 자동 영속(재시작·재실행 간 유지, SQLite 백엔드).
- `flow.plot()` — 플로우 구조를 시각화한 HTML 생성.

Flows vs Crews

Crew는 실행(자율 협업), Flow는 오케스트레이션(순서·분기·상태·실패 복구). 운영 시스템은 보통 Flow가 여러 Crew를 호출하는 구조다.

8 Tools — 내장·커스텀 도구

`crewai-tools` 패키지가 검색·스크래핑·파일·RAG·코드 실행 등 다수의 내장 도구를 제공한다. 커스텀 도구는 `@tool` 데코레이터나 `BaseTool` 상속으로 만든다.

```

from crewai.tools import tool

@tool("FX lookup")
def fx(base: str, quote: str) -> str:
    """Look up the exchange rate between two currencies."""
    return f"{base}/{quote} = 1350"

# built-in examples: from crewai_tools import SerperDevTool, FileReadTool

```

9 Memory와 Knowledge

Memory는 실행 경험을 저장·회상한다. `memory=True`로 켜며 종류는 **단기**(현재 세션 맥락, ChromaDB+RAG 임베딩), **장기**(세션 간 인사이트, SQLite), **엔티티**(인물·장소·개념, RAG)로 나뉜다. **Knowledge**는 메모리와 별개로, 텍스트·PDF·CSV 등 **외부 지식 소스**를 임베딩해 태스크 중 검색하게 하는 기능이다.

종류	저장	역할
단기 메모리	ChromaDB + RAG	현재 세션 최근 상호작용 맥락
장기 메모리	SQLite	세션 간 축적된 인사이트/결과
엔티티 메모리	RAG	인물·장소·개념 정보
Knowledge	임베딩 스토어	외부 문서(PDF/CSV/텍스트) 검색

10 실행: kickoff·입력 보간·반복·비동기

- `crew.kickoff(inputs={...})` — 실행 시작. `description`의 `{key}`가 `inputs`로 보간된다.
- `kickoff_for_each(inputs=[...])` — 입력 리스트마다 크루를 반복 실행(배치).
- `kickoff_async(...)` — 비동기 실행. 결과는 `await`로 수신.
- `result.raw` / `result.pydantic` / `result.json_dict` — 최종 산출물 접근.
`result.tasks_output`로 단계별 결과.

11 설정: YAML + CrewBase vs 순수 파이썬

에이전트·태스크를 `agents.yaml/tasks.yaml`로 선언하고 `@CrewBase` 클래스에서 `@agent/@task/@crew` 데코레이터로 묶는 것이 권장 구조다(설정과 코드 분리). `crewai create crew` CLI가 표준 프로젝트 스캐폴드를 생성한다.

```
# agents.yaml
researcher:
  role: Senior Researcher
  goal: "Collect latest facts on {topic}"
  backstory: A researcher rigorous about source verification

# crew.py
from crewai.project import CrewBase, agent, task, crew

@CrewBase
class ResearchCrew:
    agents_config = "config/agents.yaml"
    tasks_config = "config/tasks.yaml"

    @agent
    def researcher(self): return Agent(config=self.agents_config["researcher"])
    @crew
    def crew(self): return Crew(agents=self.agents, tasks=self.tasks)
```

12 학습·테스트·관측·배포

- `crew.train(n, filename)` — 사람 피드백으로 크루를 반복 학습시켜 성능 개선.
- `crew.test(n, eval_llm)` — 반복 실행으로 품질을 평가. `crewai replay`로 특정 태스크부터 재실행.
- **관측성** — `AgentOps`·`Langtrace`·`OpenLIT`·`Weave` 등과 통합해 트레이스/토큰/지연 추적.
- **배포·운영** — `CrewAI AMP`(엔터프라이즈)로 크루를 API/스케줄로 배포·모니터링.

13 CrewAI vs LangGraph 비교

항목	CrewAI	LangGraph
추상화	역할 기반 크루(고수준)	상태 그래프 노드/엣지(저수준)
핵심 모델	Agents·Tasks·Crews + Flows	State·Node·Edge + 체크포인트
제어 수준	Crew는 창발적 / Flow로 정밀 제어	그래프로 세밀한 순환·조건 제어
학습 곡선	완만(적은 코드)	가파름
상태/영속	Flow State(Pydantic) + @persist	체크포인트(InMemory/Sqlite/Postgres)
휴먼인터럽트	Task human_input	interrupt() / resume
기반	독립·경량(LangChain 비의존)	LangChain 생태계 런타임
강점	빠른 멀티 에이전트 협업·프로토타이핑	복잡 상태·감사/롤백·운영 성숙도
대표 선택	역할 협업이 명확할 때	복잡 제어·엔터프라이즈 운영

실무에선 **둘 중 하나를 고르는 문제가 아니라 적합도의 문제**다. 역할 분담이 또렷한 협업·빠른 PoC는 CrewAI가, 복잡한 분기·순환·감사 추적·롤백이 중요한 운영 시스템은 LangGraph가 유리하다. CrewAI Flows는 그 간극을 좁히려는 응답이다.

14 자주 하는 실수 모음

증상	원인	해결
Hierarchical 실행 에러	manager 미지정	manager_llm 또는 manager_agent 지정
{topic}가 그대로 출력됨	inputs 키 불일치	kickoff(inputs={"topic":...}) 키 확인
태스크가 이전 결과를 모름	context 미연결	Task(context=[prev_task])
Flow 분기가 안 됨	@router 반환값 ≠ @listen 라벨	문자열 라벨 일치시키기
메모리가 안 남음	memory=False	Crew(memory=True) + 임베더 설정
출력이 형식을 안 지킴	구조화 미설정	output_pydantic / guardrail 사용
Flow 상태가 재시작 시 소멸	@persist 미적용	@persist 데코레이터 적용

15 API 빠른 레퍼런스

목적	코드
에이전트	<code>Agent(role, goal, backstory, tools, llm)</code>
태스크	<code>Task(description, expected_output, agent, context)</code>
구조화 출력	<code>Task(output_pydantic=Model)</code>
크루	<code>Crew(agents, tasks, process, memory=True)</code>
프로세스	<code>Process.sequential Process.hierarchical</code>
실행	<code>crew.kickoff(inputs={...}) / kickoff_for_each</code>
플로우	<code>Flow[State], @start / @listen / @router</code>
상태 영속	<code>@persist</code>
커스텀 도구	<code>@tool("name") def f(...) -> str</code>
프로젝트 생성	<code>crewai create crew</code>

CrewAI는 빠르게 진화합니다. 본 문서는 2026-06 시점 기준이며, 정확한 시그니처는 docs.crewai.com과 설치된 버전 릴리스 노트로 최종 확인하세요.

정답·해설은 다음 장에 있습니다. 보기 중 하나를 고르세요.

Q1. CrewAI의 가장 핵심적인 추상화는?

- A. 상태 그래프의 노드와 엣지
- B. 역할(role) 기반 에이전트들의 크루(Crew)
- C. 벡터 인덱스
- D. 프롬프트 템플릿 체인

Q2. CrewAI의 두 빌딩블록을 바르게 짝지은 것은?

- A. Nodes와 Edges
- B. Chains와 Tools
- C. Crews와 Flows
- D. Agents와 Embeddings

Q3. Agent의 페르소나를 정의하는 세 요소는?

- A. role, goal, backstory
- B. name, id, token
- C. input, output, schema
- D. start, listen, router

Q4. Crew에서 태스크 실행 방식을 결정하는 것은?

- A. Embedder
- B. Process
- C. Router
- D. Checkpointer

Q5. Hierarchical 프로세스에서 반드시 필요한 것은?

- A. manager_llm 또는 manager_agent
- B. memory=True
- C. @persist
- D. output_pydantic

Q6. Sequential 프로세스의 특징은?

- A. 매니저가 동적 위임
- B. 태스크를 정의 순서대로 실행
- C. 이벤트 기반 분기
- D. 병렬 무작위 실행

Q7. 이전 태스크의 결과를 다음 태스크 입력으로 연결하는 필드는?

- A. tools
- B. context
- C. backstory
- D. guardrail

Q8. 태스크 산출물을 구조화(스키마 강제)하려면?

- A. allow_delegation=True
- B. output_pydantic=Model
- C. verbose=True
- D. process=hierarchical

Q9. Flow의 진입점을 표시하는 데코레이터는?

- A. @router
- B. @listen
- C. @start
- D. @crew

Q10. Flow에서 이전 단계 완료 시 실행을 트리거하는 데코레이터는?

- A. @start
- B. @listen
- C. @tool
- D. @agent

Q11. Flow에서 반환 문자열로 경로를 분기하는 데코레이터는?

- A. @router
- B. @persist
- C. @listen
- D. @task

Q12. Flow의 공유 State는 보통 무엇으로 정의하나?

- A. 전역 변수
- B. Pydantic 모델
- C. SQL 테이블
- D. JSON 문자열만

Q13. Flow 상태를 재시작/재실행 간 유지하려면?

- A. @persist
- B. verbose=True
- C. allow_delegation
- D. Process.sequential

Q14. 크루 실행을 시작하고 {placeholder}를 보간하는 호출은?

- A. crew.run()
- B. crew.kickoff(inputs={...})
- C. crew.compile()
- D. crew.invoke()

Q15. 입력 리스트마다 크루를 반복 실행하는 메서드는?

- A. kickoff_for_each
- B. map_reduce
- C. astream
- D. replay

Q16. CrewAI의 단기 메모리는 주로 무엇으로 구현되나?

- A. SQLite
- B. ChromaDB + RAG
- C. Redis 큐
- D. 파일 로그

Q17. Memory와 Knowledge의 차이로 옳은 것은?

- A. 둘은 동일하다
- B. Memory=실행 경험 저장/회상, Knowledge=외부 문서 임베딩 검색
- C. Knowledge가 단기 메모리다
- D. Memory는 PDF 전용

Q18. 설정과 코드를 분리하는 권장 구조는?

- A. 모든 것을 한 함수에
- B. agents.yaml/tasks.yaml + @CrewBase
- C. 환경변수만 사용
- D. JSON-RPC 서버

Q19. CrewAI와 LangGraph 비교로 가장 정확한 것은?

- A. CrewAI=저수준 그래프, LangGraph=고수준 역할
- B. CrewAI=역할 기반 고수준, LangGraph=상태 그래프 저수준
- C. 둘 다 LangChain 필수
- D. 둘은 기능이 동일

Q20. 커스텀 도구를 만드는 방법은?

- A. @tool 데코레이터 또는 BaseTool 상속
- B. @router
- C. Process.hierarchical
- D. output_json

A

정답 및 해설

Q1 정답: B

CrewAI의 핵심은 역할(role) 기반 에이전트들을 크루(Crew)로 묶어 협업시키는 직관적 추상화.

Q2 정답: C

두 빌딩블록은 Crews(자율 협업 실행)와 Flows(이벤트 기반 오케스트레이션).

Q3 정답: A

Agent는 role·goal·backstory로 페르소나를 정의하고, 이 서술이 프롬프트로 들어가 행동을 좌우한다.

Q4 정답: B

Crew의 process(Sequential/Hierarchical)가 태스크 실행 방식을 결정한다.

Q5 정답: A

Hierarchical은 매니저 에이전트가 위임하므로 manager_llm 또는 manager_agent 지정이 필수.

Q6 정답: B

Sequential은 태스크를 정의된 순서대로 차례로 실행한다.

Q7 정답: B

Task(context=[prev_task])로 선행 태스크 결과를 다음 태스크 입력으로 연결한다.

Q8 정답: B

output_pydantic(또는 output_json)으로 산출물 스키마를 강제한다.

Q9 정답: C

@start()가 Flow의 진입점을 표시한다.

Q10 정답: B

@listen(이전단계)는 선행 단계 완료 시 해당 메서드를 실행한다.

Q11 정답: A

@router()는 반환 문자열을 라벨로 사용해 다음 경로를 분기한다.

Q12 정답: B

Flow의 공유 State는 보통 Pydantic 모델로 정의하며 모든 단계가 읽고 쓴다.

Q13 정답: A

@persist 데코레이터로 State를 자동 영속해 재시작/재실행 간 유지한다(SQLite 백엔드).

Q14 정답: B

crew.kickoff(inputs={...})로 실행을 시작하고 description의 {placeholder}를 보간한다.

Q15 정답: A

kickoff_for_each(inputs=[...])가 입력 리스트마다 크루를 반복 실행(배치)한다.

Q16 정답: B

단기 메모리는 ChromaDB + RAG 임베딩으로 현재 세션 맥락을 유지한다. 장기 메모리는 SQLite.

Q17 정답: B

Memory는 실행 경험을 저장/회상, Knowledge는 외부 문서(PDF/CSV/텍스트)를 임베딩해 검색하게 하는 별개 기능.

Q18 정답: B

agents.yaml/tasks.yaml로 선언하고 @CrewBase 클래스의 @agent/@task/@crew로 묶는 것이 권장 구조.

Q19 정답: B

CrewAI=역할 기반 고수준 추상화, LangGraph=상태 그래프 저수준 제어. CrewAI는 LangChain 비의존.

Q20 정답: A

커스텀 도구는 @tool 데코레이터 또는 BaseTool 상속으로 만든다.